

WEEK 1 TASK

Project Planning & System Architecture

Campus Placement Management System

A Full-Stack Web Application for Streamlining Campus Recruitment

Junior Full Stack Developer Intern

Internship Period: 09 September 2026 - 21 October 2026

Week 1 Deliverable: Project brief, flowcharts, wireframes, technical architecture, database schema
and design rationale

Prepared by: Bhavani

GitHub Repository: <https://github.com/Bhavani160/campus-placement-management-system>

1. Project Brief

Overview. The Campus Placement Management System (CPMS) is a full-stack web application that centralizes campus recruitment activities. Students can maintain profiles, discover placement opportunities, check basic eligibility, submit applications and track status. Recruiters can manage company information and job postings, while administrators can oversee users, companies, postings and applications.

Problem Statement. Placement information is commonly distributed through spreadsheets, notices, messaging groups and separate forms. This can cause missed opportunities, duplicate records, difficulty verifying eligibility and limited visibility into application progress. CPMS addresses these issues through a single structured platform.

Proposed Solution. A responsive web frontend communicates with a RESTful backend API. The backend manages authentication, authorization, validation and business logic, while a relational database stores persistent placement data.

Target Users. Students; recruiters/company representatives; placement administrators.

Objectives.

1. Centralize placement opportunities and student profiles.
2. Automate basic eligibility checking.
3. Provide a traceable application workflow.
4. Reduce manual placement administration.
5. Demonstrate practical full-stack planning, system design and documentation.

2. Features & Requirements

Core Features

Area	Features
Student	Registration/login, profile, academic details, skills, job search, eligibility, applications, status tracking
Recruiter	Company profile, job creation/editing, eligibility criteria, applicant viewing, status updates
Admin	User/company management, job-post approval, application monitoring and placement records
Platform	Role-based access, validation, search/filtering, status messages and audit-friendly records

Functional Requirements

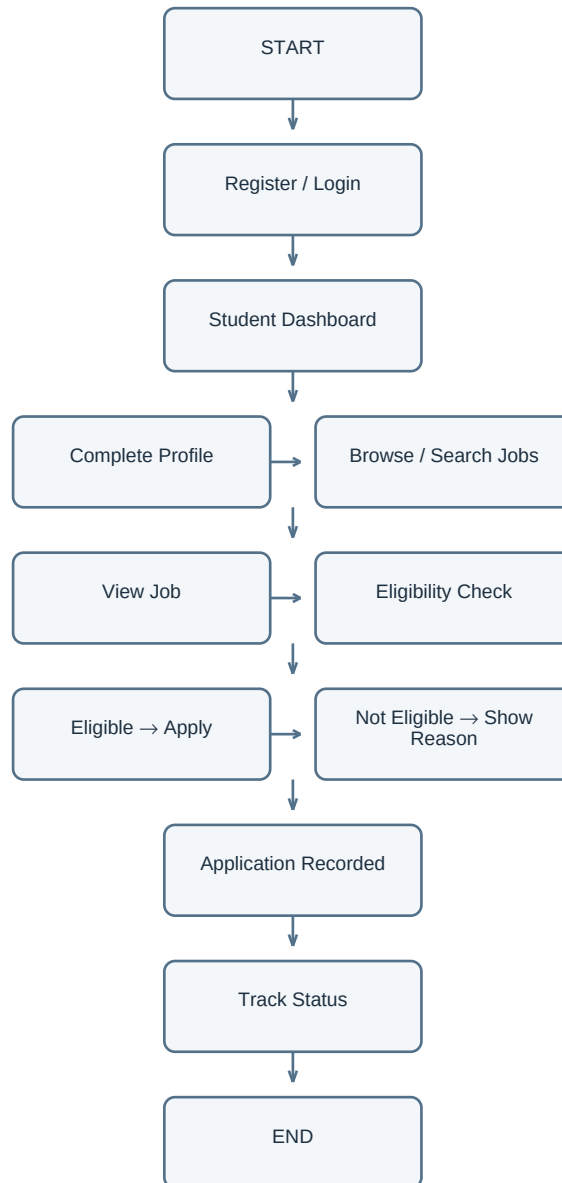
ID	Requirement
FR-01	Users shall register and authenticate securely.
FR-02	The system shall support Student, Recruiter and Admin roles.
FR-03	Students shall create and update their profiles.
FR-04	Recruiters shall create, edit and manage job postings.
FR-05	Students shall browse, search and filter job postings.
FR-06	The system shall perform basic eligibility checks.
FR-07	Eligible students shall be able to apply.
FR-08	Students shall track application status.
FR-09	Recruiters shall view applicants and update status.
FR-10	Admins shall manage users, companies and postings.

Non-Functional Requirements: Security, performance, usability, maintainability, scalability and reliability. The application should validate inputs on the server, enforce authorization, use efficient database queries, provide a responsive UI and maintain a modular codebase.

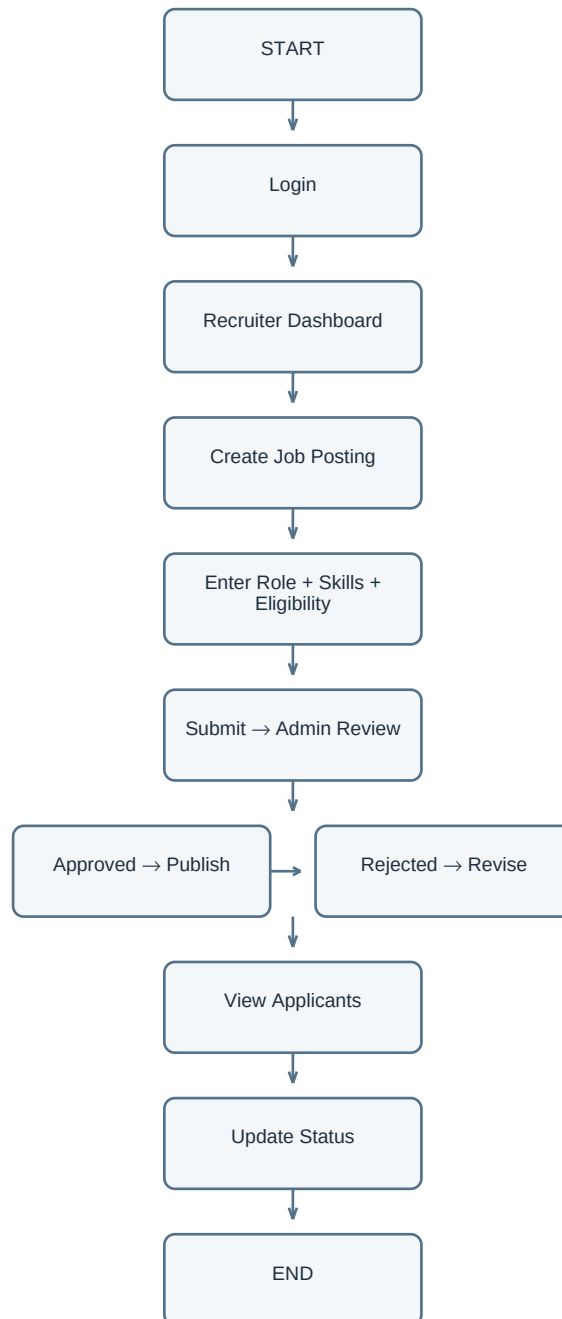
3. User Interaction Flowcharts

The following flows describe the primary user journeys and decision points required by the application.

Student Flow

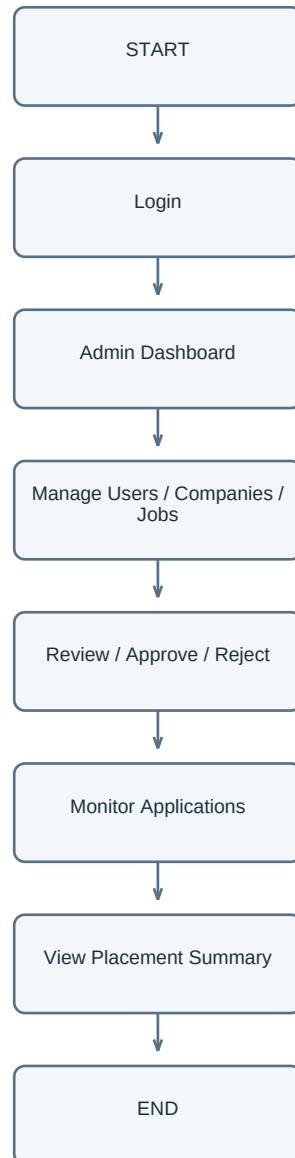


Recruiter Flow



4. Admin Workflow

Administrator Flow



5. Front-End Wireframes

These low-fidelity wireframes define layout, information hierarchy and primary controls. They are implementation-oriented and can be refined visually during frontend development.

Wireframe A — Login / Registration

CPMS

SearchNotificationsProfile

LOGIN / REGISTER

Email

Password

[LOGIN] [REGISTER]

Wireframe B — Student Dashboard

CPMS

SearchNotificationsProfile

DASHBOARD

Jobs

Applications

Profile

Logout

Welcome, Student

Profile: 80%

Recommended Jobs

3 new matches

RECENT APPLICATIONS

Software Developer | Applied

Data Analyst | Shortlisted

Web Developer | Pending

6. Front-End Wireframes (continued)

Wireframe C — Job Details & Application

CPMS

SearchNotificationsProfile

JOB TITLE | COMPANY | LOCATION

Description and responsibilities
Required skills: JavaScript, React, Node.js
Eligibility: CGPA >= 7.0 | Deadline: 30/09/2026

Eligibility: ELIGIBLE

[APPLY NOW] [BACK TO JOBS]

Wireframe D — Admin Dashboard

CPMS

SearchNotificationsProfile

ADMIN

Overview
Users
Companies
Job Posts
Applications

Users

Jobs

Applications

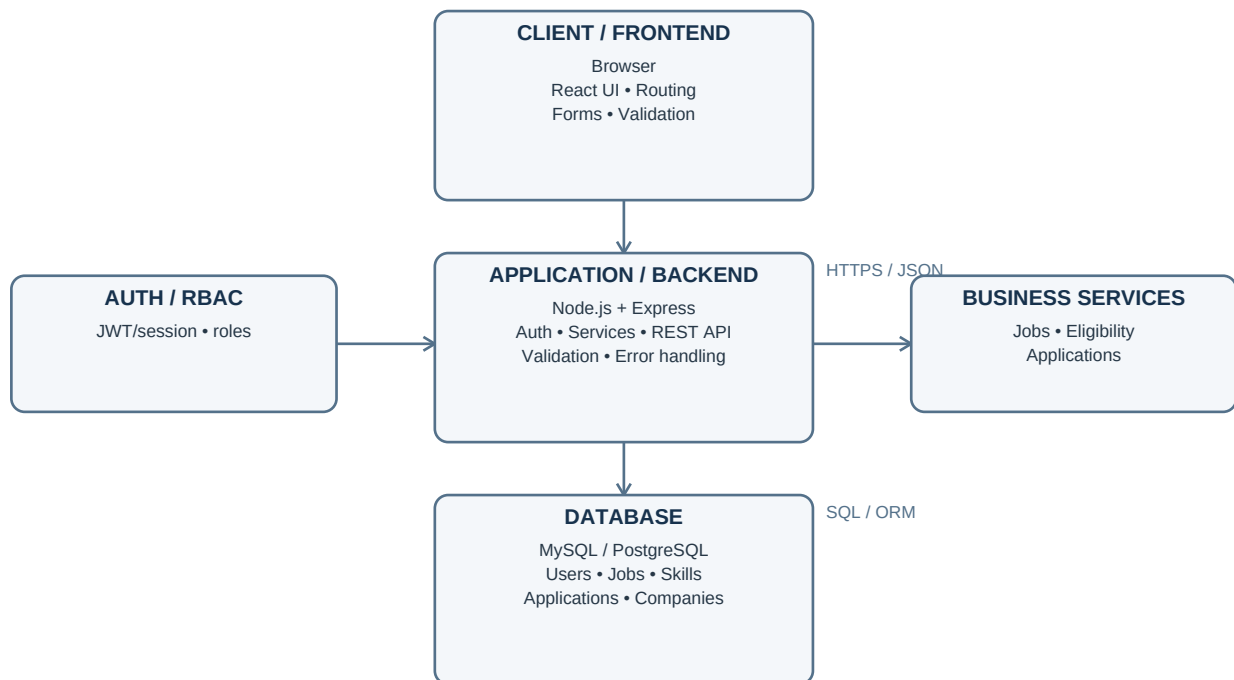
1,25048420

JOB POSTS / APPLICATIONS TABLE

Search | Filter | Approve | Reject | Edit

7. Technical Architecture

Architecture Style: Three-tier client-server architecture with a REST API. Presentation, business logic and persistence are separated to improve maintainability, testing and scalability.



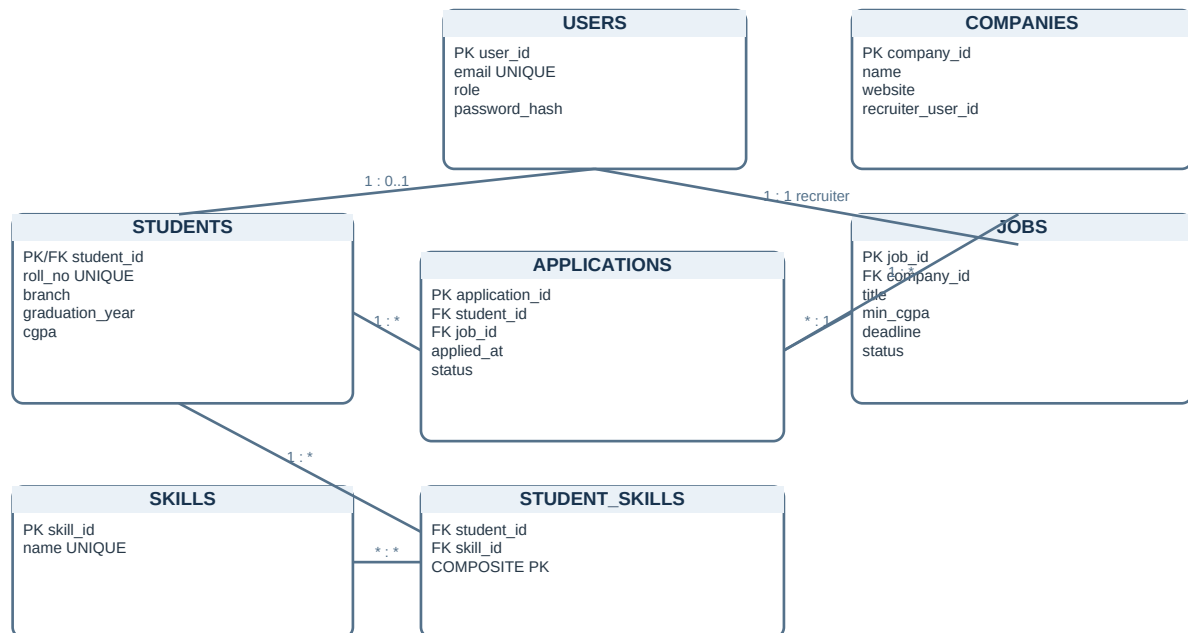
Data flow: User interaction occurs in the React frontend. Requests are sent over HTTPS as REST/JSON to the Node.js/Express backend. The backend authenticates the user, applies role permissions and business rules, then reads/writes data through the database layer.

Deployment considerations: Frontend and backend may be deployed independently. Production secrets belong in environment variables. Database access should be private to the backend service and production traffic should use HTTPS.

8. API Endpoint Design

Method	Endpoint	Purpose
POST	/api/auth/register	Create a user account
POST	/api/auth/login	Authenticate a user
GET	/api/jobs	List, search and filter jobs
GET	/api/jobs/:id	View job details
POST	/api/jobs	Create job posting (Recruiter)
PUT	/api/jobs/:id	Update job posting
POST	/api/jobs/:id/apply	Submit a student application
GET	/api/applications/me	View current student's applications
GET	/api/jobs/:id/applicants	View applicants (Recruiter)
PATCH	/api/applications/:id/status	Update application status

9. Database Schema & Relationships



Relationship summary: One company publishes many jobs; one job receives many applications; one student can submit many applications; students and skills form a many-to-many relationship through STUDENT_SKILLS. User accounts provide identity and role information.

10. Database Definition

Entity	Key Fields
USERS	user_id PK, name, email UNIQUE, password_hash, role, created_at
STUDENTS	student_id PK/FK, roll_no UNIQUE, branch, graduation_year, cgpa
COMPANIES	company_id PK, name, description, website, recruiter_user_id FK
JOBS	job_id PK, company_id FK, title, description, location, min_cgpa, deadline, status
APPLICATIONS	application_id PK, job_id FK, student_id FK, applied_at, status
SKILLS	skill_id PK, name UNIQUE
STUDENT_SKILLS	student_id FK, skill_id FK, composite primary key

11. Design Decisions & Rationale

React: component-based UI supports reusable dashboards, forms and job cards.

Node.js + Express: provides a straightforward REST API and allows JavaScript across the stack.

Relational database: placement data has strong relationships and requires consistent updates, making SQL appropriate.

REST/JSON: creates a clean client-server boundary and supports independent API testing.

Role-based authorization: prevents students, recruiters and admins from accessing unrelated operations.

Layered backend: separates routing, business logic and data access for maintainability.

12. Security & Validation

Passwords should be stored using a strong salted hashing algorithm. Protected routes should enforce role-based authorization. Client-side validation improves usability, while server-side validation remains authoritative. Database operations should use parameterized queries or ORM protections. Secrets must never be committed to GitHub. Production deployment should use HTTPS, secure headers, rate limiting, controlled error messages and appropriate logging.

13. Scope & Future Enhancements

Initial scope: authentication, profiles, job postings, eligibility, applications, status tracking and admin management.

Future enhancements: email notifications, resume parsing, interview scheduling, analytics dashboards, recommendation engine, placement statistics and college identity integration.

14. Development Plan

Phase	Planned Work
1. Planning	Requirements, roles, architecture, wireframes and schema
2. Backend	Authentication, models, REST APIs and validation
3. Frontend	Responsive pages, forms, dashboards and API integration
4. Integration	End-to-end eligibility and application workflows
5. Testing	API, UI, validation, authorization and edge-case testing
6. Deployment	Configuration, documentation and GitHub repository

15. Conclusion

The Campus Placement Management System provides a realistic full-stack software scenario with clearly defined users, workflows, APIs and relational data. The proposed design establishes a maintainable technical foundation for implementation, testing and deployment. Subsequent development should keep the GitHub repository synchronized with the architecture documented here.